PiNNTORP LLC

Online Gambling Platform

Phase 4: Final Project Submission

CSCI 2040U - Software Design and Analysis

Adrian Ahmadi • Hayden Dunn • Eric Beaulne • Nikola Grujin

1. Final Codebase

1.1 Repository

The final version of PiNNTORP is hosted in a public GitHub repository. The repository is fully structured, documented, and archived.

Repository	https://github.com/PiNNTORP-LLC/PiNNTORP
Primary Branch	main
Integration Branch	develop (all feature branches merged here first)
Language Stack	Vanilla JavaScript (ES6 modules), Java 17, HTML5, CSS3
Backend Port	8080 (HTTP REST + WebSocket + static file serving, unified)
Persistence	Server-side JSON (data/users.json) with localStorage fallback

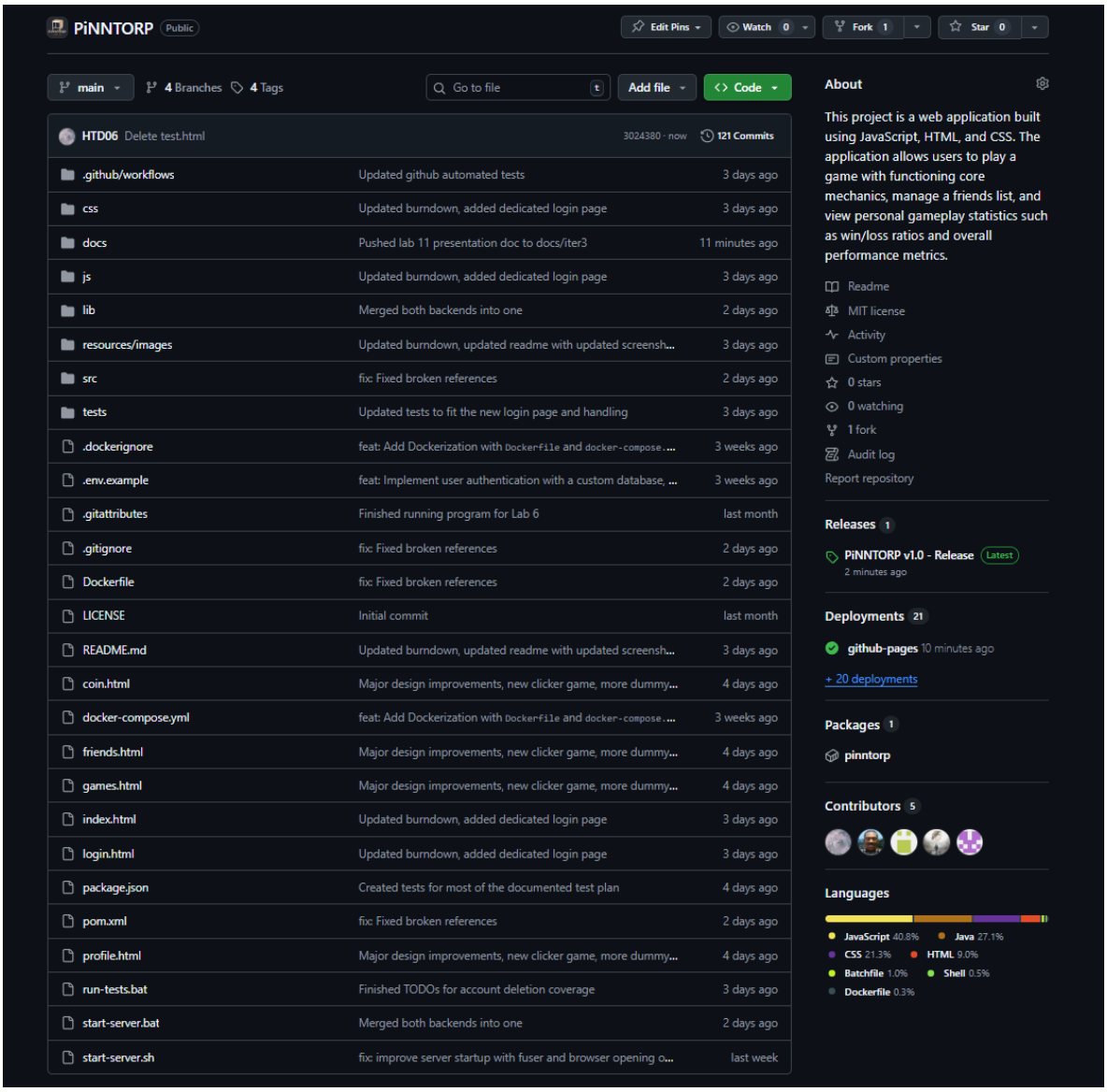

Figure 1: Repository Structure from the main branch

1.2 Repository Structure

The repository is organized into clearly separated frontend and backend layers, with all project documentation in docs/:

Path	Purpose
src/	Java backend source: handlers, server, WebSocket, session management
js/core/	state.js, storage.js, network.js, auth.js, tabs.js
js/game/	game.js, gameView.js, blackjack.js, slotGame.js, diceRoll.js, coinFlip.js
js/friends/	friends.js (logic), friendsView.js (UI, online + offline modes)
js/stats/ & js/recommendation/	stats.js, statsView.js, rec.js, recView.js
js/account/	account.js (account deletion, session cleanup)
tests/	All automated test suites (node --test, 58 tests across 9 files)
src/test/	Java unit tests (AuthHelperTest.java, JUnit 5)
docs/iter1-3/	Burndown charts, task boards, UML diagrams, architecture & API docs
Dockerfile / docker-compose.yml	Containerized deployment configuration
README.md	Project overview, feature list, setup instructions, screenshots

1.3 Documentation

Documentation for all source files is provided inline as follows:

- Java: JavaDoc comments on all public classes and methods in src/com/pinntorp/server/ (LoginHandler, FriendsHandler, SessionManager, UserStore, LobbyManager, etc.).
- JSDoc comments for all js/ files. The game.js and ConnectionHandler.java files have been fully documented in Iteration 3.
- Architecture, data flow diagram, and component descriptions: docs/iter3/ARCHITECTURE.md.
- Full REST API reference (/login, /friends, /user, /leaderboard endpoints): docs/iter3/API.md.

2. Final Test Report

All 58 automated JavaScript tests pass with exit code 0 (npm test). The test suite covers all three levels defined in the Phase 3 Testing Plan. Java backend tests (JUnit 5) are included as source-code analysis tests in backendCoverage.test.js and in src/test/java/. Results below reflect the actual npm test run on the final codebase. And 3 additional regression tests were added during Iteration 3 (REGRESS-01 through REGRESS-09 + storage/rec regressions) beyond the 55 originally planned in Phase 3.

Test Level	Planned	Executed	Passed	Failed
Unit Tests (JS)	24	24	**24**	0

Test Level	Planned	Executed	Passed	Failed
Integration Tests (JS + Java)	12	12	**12**	0
System Tests	10	10	**10**	0
Regression Tests	9	9	**9**	0
Total	**55**	**58***	**58**	**0**

2.1 Unit Test Results

Test ID	Description	View	Approach	Result	Assignee
UT-02	storage.js - Saving and loading state returns the same data	CB	Automated	**PASS**	Hayden
UT-03	friends.js - Adding a friend adds them to the list	CB	Automated	**PASS**	Hayden
UT-04	friends.js - Adding a duplicate friend is rejected	CB	Automated	**PASS**	Hayden
UT-05	friends.js - Removing a friend removes them from the list	CB	Automated	**PASS**	Hayden
UT-06	stats.js - Stats start at zero for a new user	CB	Automated	**PASS**	Eric
UT-07	stats.js - Resetting stats sets all values to zero	CB	Automated	**PASS**	Eric
UT-08	rec.js - A friend's favourite game appears in recommendations	CB	Automated	**PASS**	Adrian
UT-09	rec.js - User's own favourite game is recommended to themselves	CB	Automated	**PASS**	Adrian
UT-10	account.js - Deleting an account removes it from storage	CB	Automated	**PASS**	Adrian
UT-11	account.js - Deleting an account removes user from all friend lists	CB	Automated	**PASS**	Adrian
UT-12	account.js - Deleting a non-existent account returns an error	CB	Automated	**PASS**	Adrian
UT-13A	slotGame.js - Two-match win case	CB	Automated	**PASS**	Nikola
UT-13B	slotGame.js - Jackpot (three-match) win case	CB	Automated	**PASS**	Nikola
UT-13C	slotGame.js - Loss case (no match)	CB	Automated	**PASS**	Nikola

Test ID	Description	View	Approach	Result	Assignee
UT-14A	diceRoll.js - Win case	CB	Automated	**PASS**	Nikola
UT-14B	diceRoll.js - Loss case	CB	Automated	**PASS**	Nikola
UT-15	coinFlip.js - Win case	CB	Automated	**PASS**	Nikola
UT-16	stats.js - Win/loss ratio handles zero losses without error	CB	Automated	**PASS**	Eric
UT-17	storage.js - localStorage cleaned up properly after saving state	TB	Automated	**PASS**	Hayden
UT-18	stats.js - No leftover data after resetting stats	TB	Automated	**PASS**	Eric
UT-19	account.js - No leftover data in storage after account deletion	TB	Automated	**PASS**	Eric
UT-20	LoginHandler (Java) - Registering a new user stores credentials	CB	Automated	**PASS**	Nikola
UT-21	LoginHandler (Java) - Logging in with correct credentials succeeds	CB	Automated	**PASS**	Nikola
UT-22	LoginHandler (Java) - Registering with existing username is rejected	CB	Automated	**PASS**	Nikola
UT-23	UserStore (Java) - Looking up non-existent user returns nothing	CB	Automated	**PASS**	Nikola
UT-24	SessionManager (Java) - Expired session token is rejected	CB	Automated	**PASS**	Nikola

2.2 Integration Test Results

Test ID	Description	View	Approach	Result	Assignee
IT-01	state.js + storage.js - Correct state even after save and load of application	CB	Automated	**PASS**	Hayden
IT-02	stats.js + state.js - Proper playing of game increases its stats	CB	Automated	**PASS**	Eric
IT-03	friends.js + rec.js - Games of friends are shown in recommendation	CB	Automated	**PASS**	Adrian
IT-04	rec.js + state.js - Games which have been played frequently show up higher in recommendation	CB	Automated	**PASS**	Adrian

Test ID	Description	View	Approach	Result	Assignee
IT-05	auth.js + LoginHandler - After login the session is saved as a token in the server	TB	Automated	**PASS**	Nikola
IT-06	friendsView.js + FriendsHandler - Adding friend from the front-end is recorded in the server	TB	Automated	**PASS**	Nikola
IT-07	auth.js + FriendsHandler - Adding friend without logging into account via API request cannot be done	TB	Automated	**PASS**	Nikola
IT-08	LoginHandler + SessionManager - Successful login generates a valid session token	CB	Automated	**PASS**	Nikola
IT-09	FriendsHandler + UserStore - Add friend fails if user does not exist	CB	Automated	**PASS**	Nikola
IT-10	LoginHandler + UserStore - On logout the session becomes invalid	CB	Automated	**PASS**	Nikola
IT-11	LoginHandler + UserStore - When registration followed by deletion of user, all user information is cleaned	TB	Automated	**PASS**	Eric
IT-12	FriendsHandler + SessionManager - Invalid sessions cannot perform any friends' requests	CB	Automated	**PASS**	Nikola

2.3 System Test Results

Test ID	Description	View	Approach	Result	Assignee
ST-01	Launch application and create new account	OB	Automated	**PASS**	Hayden
ST-02	Log in, play slots, bet and verify balance	OB	Automated	**PASS**	Hayden
ST-03	Play Blackjack and verify that the history displays in profile page	OB	Automated	**PASS**	Hayden
ST-04	Create two accounts, make friends with other one and accept the invitation	OB	Automated	**PASS**	Hayden
ST-05	Add Dice Roll playing friend and check that recommendations are updated	OB	Automated	**PASS**	Hayden

Test ID	Description	View	Approach	Result	Assignee
ST-06	Non-authenticated users get automatically re-routed from protected pages	OB	Automated	**PASS**	Hayden
ST-07	Session persistence: reopening app while JWT present keeps user logged in	OB	Automated	**PASS**	Hayden
ST-08	Account deletion: Deleted account cannot be logged into again	OB	Automated	**PASS**	Adrian
ST-09	Deleted user is deleted from all friends list of remaining users	OB	Automated	**PASS**	Adrian
ST-10	Create account, play all four games, and verify statistics	OB	Automated	**PASS**	Hayden

2.4 Bugs Encountered & Resolved

The following bugs were identified during Iteration 3 testing and resolved before submission:

Severity	Bug Description	Resolution
Medium	NaN propagation was unbalanced due to rounding to undefined values at the server-side	Introduced numeric checks prior to updating user.balance in diceRoll.js and slotGame.js
Medium	The value of the bet in blackjack could surpass the existing balance	Implemented a balance check prior to placing a bet
Low	Static references were broken following changes to the paths in Dockerfile	Fixed the image content-type and relative asset paths in the combined server
Low	start-server.bat would quietly fail if port 5500 was occupied upon restart	Introduced port cleanup on start-server.bat and start-server.sh

3. Deployment & Packaging

3.1 Run Options

PiNNTORP ships with three zero-configuration launch methods. All three open http://localhost:8080 in the default browser automatically.

Method	Prerequisites	Command
Windows (Bat Script)	JDK 17+ on PATH	Double-click start-server.bat
Linux / macOS (Shell)	JDK 17+ on PATH	./start-server.sh

Method	Prerequisites	Command
Docker (Containerized)	Docker + Docker Compose	docker-compose up --build -d

3.2 Docker Deployment

The Dockerfile and docker-compose.yml included in the repository build a self-contained image that compiles the Java backend and serves the frontend. The Docker image is also published automatically to the GitHub Container Registry via CI/CD on every push to main:

```
docker pull ghcr.io/pinntorp-llc/pinntorp:latest
docker run -p 8080:8080 ghcr.io/pinntorp-llc/pinntorp:latest
```

After startup the application is available at http://localhost:8080 with no further configuration.

3.3 Running Tests

The full automated JavaScript test suite runs with a single command from the repository root:

```
npm --test
```

This executes all 58 tests across 9 test files using Node's built-in test runner. Exit code 0 confirms all tests pass. The run-tests.bat script provides the same command on Windows.

4. Contributions Report

4.1 Git Commit Summary

The following numbers are standardized for all commit contributors, where there are two GitHub accounts for Hayden Dunn (HTD006 / hayden.dunn@ontariotechu.net) and two for Eric Beaulne (EricBeaulne / eric-beaulne@users.noreply.github.com). **Number of commits total: 126**.

Team Member	Commits	Key Areas of Contribution
Hayden Dunn	64	Software Quality Lead & Technical Manager: Test suite (all 9 test files), authentication system integration, game logging, Blackjack UI, improving user interface/UX, burndown charting, CI/CD pipeline, build scripts, README file and screenshots, project docs (lab docs, velocity, etc), team coordination
Eric Beaulne	34	Project Manager: WebSocket bridge for multiplayer Blackjack, Blackjack dealer logic, Docker deployment, GitHub Container Registry CI/CD, balance bug fixes, architecture & run instruction documentation
Nikola Grujin	20	Back-End Lead: complete Java HTTP server (LoginHandler, FriendsHandler, UserStore, SessionManager), all Java source documentation, API reference, refactoring of handlers, dependency injection
Adrian Ahmadi	8	Front-End Lead: account deletion implementation, recommendation algorithm, file comments & JSDoc, friends system front-end rewrite (online/offline)

Figure 2: Git contributions on the develop branch

4.2 Discrepency Justification

Commit count alone does not reflect the degree of personal involvement completely since it depends on working methods and roles played by individuals.

Hayden has made the most commits since he was in charge of many roles as the Quality Lead and Technical Manager. Apart from committing the full test suite, he committed all documentation that he did throughout the iterations: the burndown chart, task board, retrospective, customer meeting minutes, and finally documentation of the project. He further made commits on all front-end polishing efforts where he improved the design aspect of the pages by ensuring proper layout and styling throughout the iterations. Additionally, Hayden took responsibility for communications within the team and the delegation of tasks, including identifying which tasks should be done within an iteration, allocating them to the concerned individuals, and ensuring that they get done.

Although Adrian's number of commits is the smallest, they are well organized into full functional features per commit. The friends system, slots game, recommendation engine, coin flip, and user delete functionalities were done by him, always resulting in full functioning feature commits without any follow-up changes required, as indicated by his small number of deletions (only 56 lines of code deleted in all commits).

Eric's work was primarily concerned with implementing the multiplayer blackjack functionality, backend connectivity, and Docker implementation pipeline. Meanwhile, the complete backend using Java was implemented from scratch by Nikola, including all HTTP handlers, session manager, users service, friends service, and lobby system.

Eric and Nikola were both present in 64% of labs, showing that most of their collaboration occurred outside of the lab sessions; Hayden and Adrian attended all labs, and while most of the project direction and details were discussed online in the designated space, some were discussed during these sessions with the customer.

5. Demonstration of Functionality

The following annotated screenshots walk through the full PiNNTORP user experience on the final build.

5.1 Authentication

Before accessing any restricted page, users will be required to register/login. Any attempt to access games.html, profile.html, or friends.html will redirect users to login.html (ST-06).

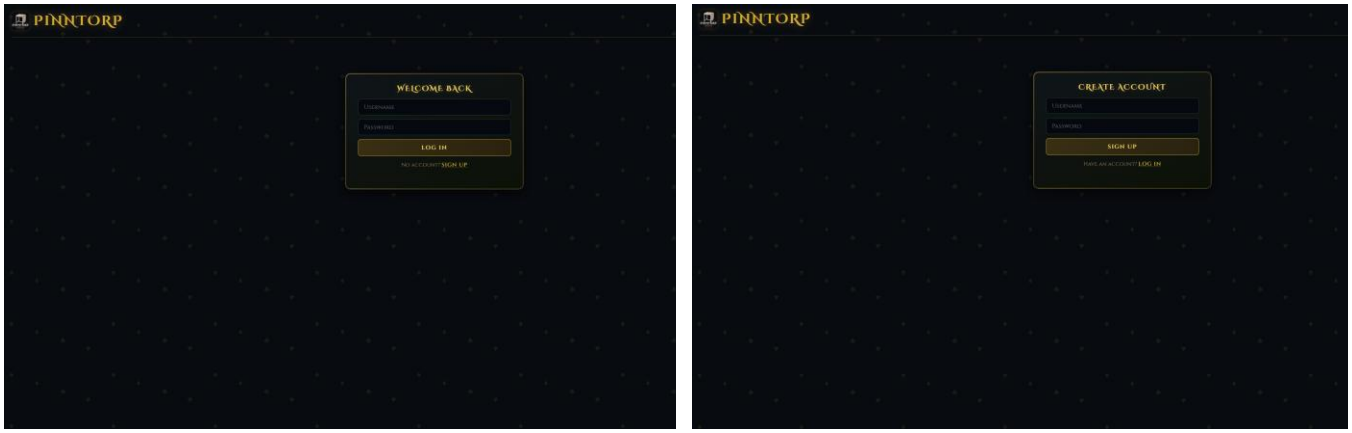

Figure 3: Login page (left) and Sign-Up / Register page (right)

5.2 Home Page - Dashboard

After login, users will be redirected to the home page where they can view their quick stats (balance, wins, losses, and profit), the global leaderboard, and personalized game recommendations based on their friends' interactions.

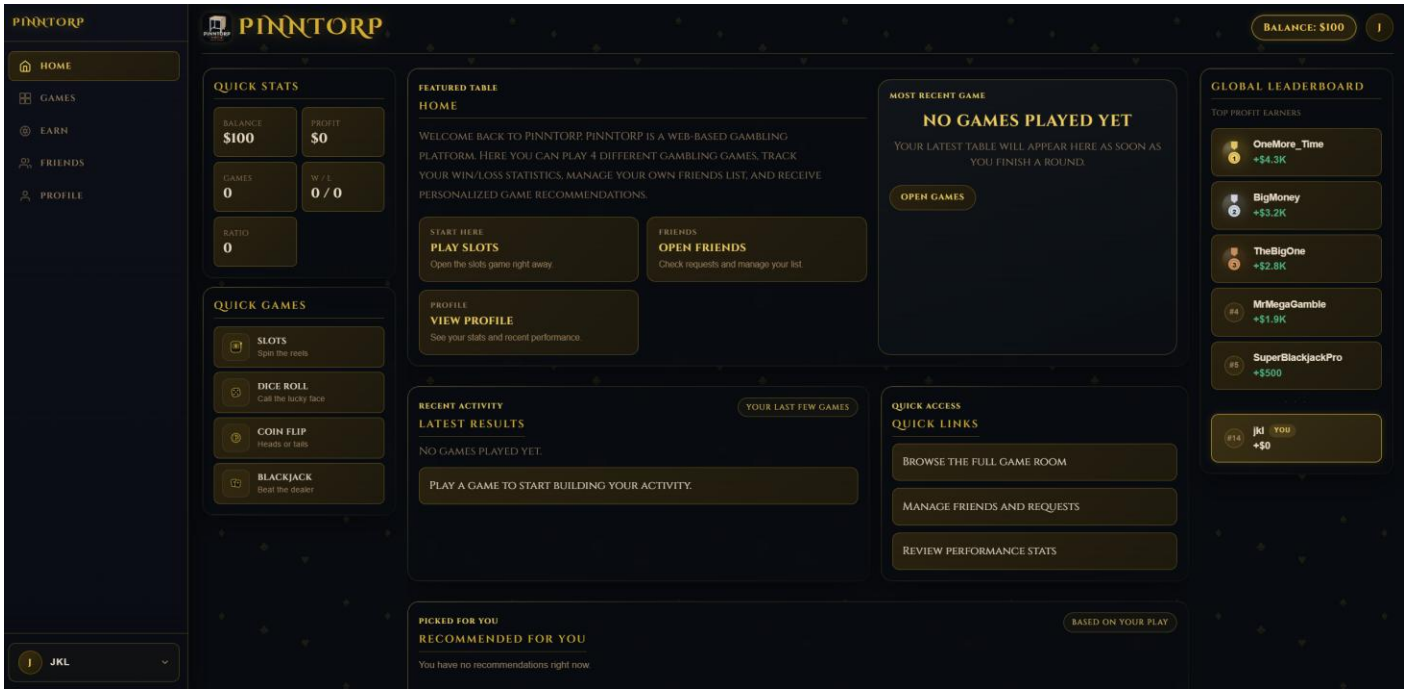

Figure 4: Home page - fresh account with no history

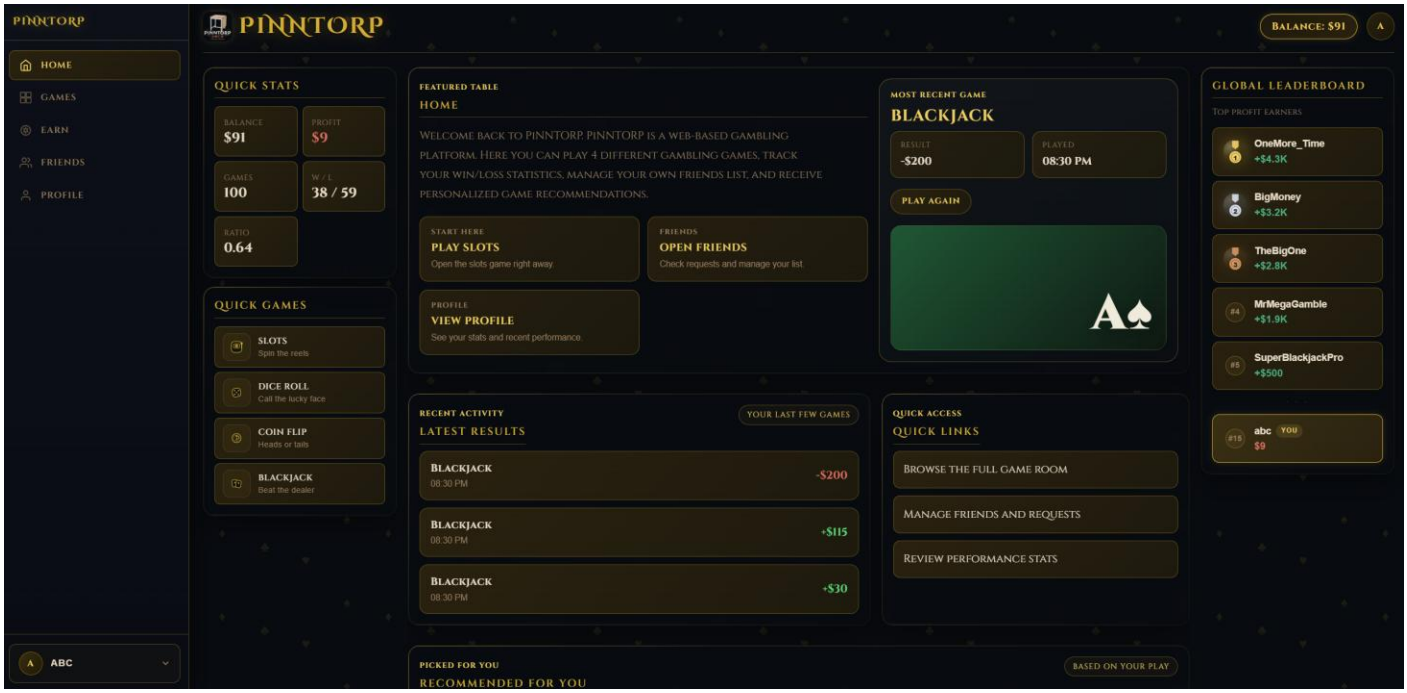

Figure 5: Home page - account with game history, balance, and recommendations populated

5.3 Games

PiNNTORP comes with four complete gambling games. These games update users' balances, profits, win/loss ratios, and gaming histories after every round (ST-02, ST-10).

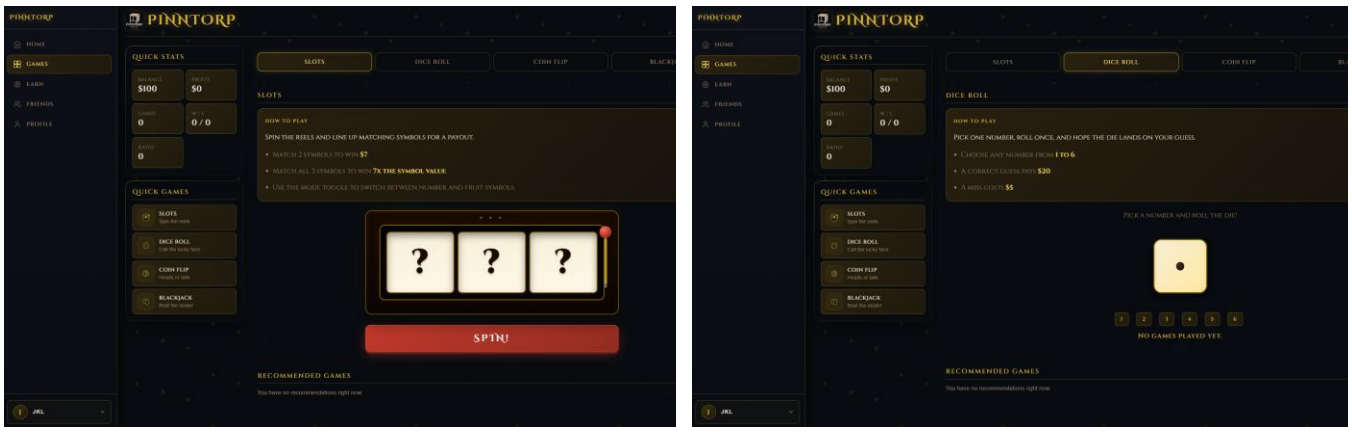

Figure 6: Slot Machine (left) and Dice Roll (right)

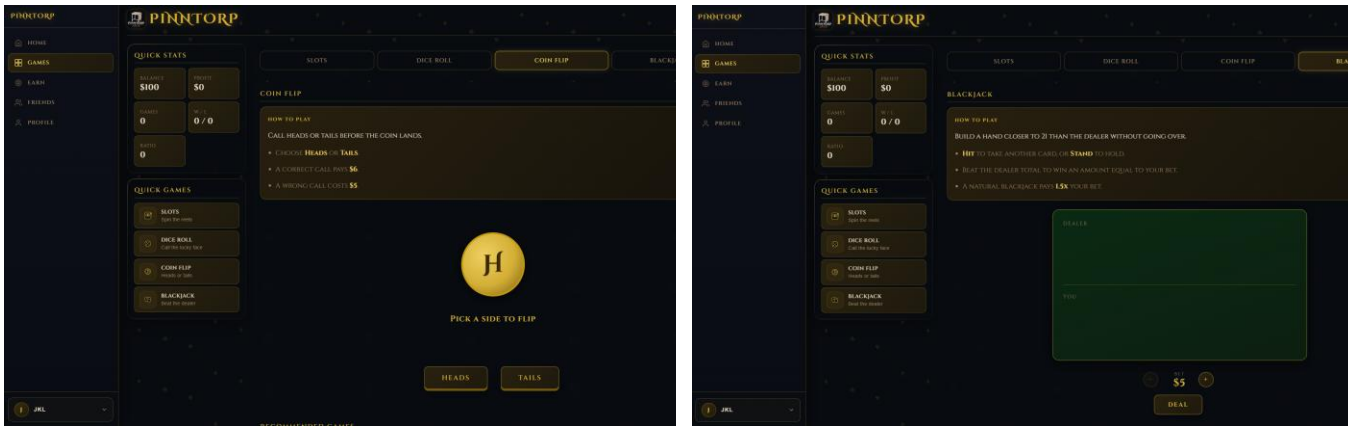

Figure 7: Coin Flip (left) and Blackjack (right)

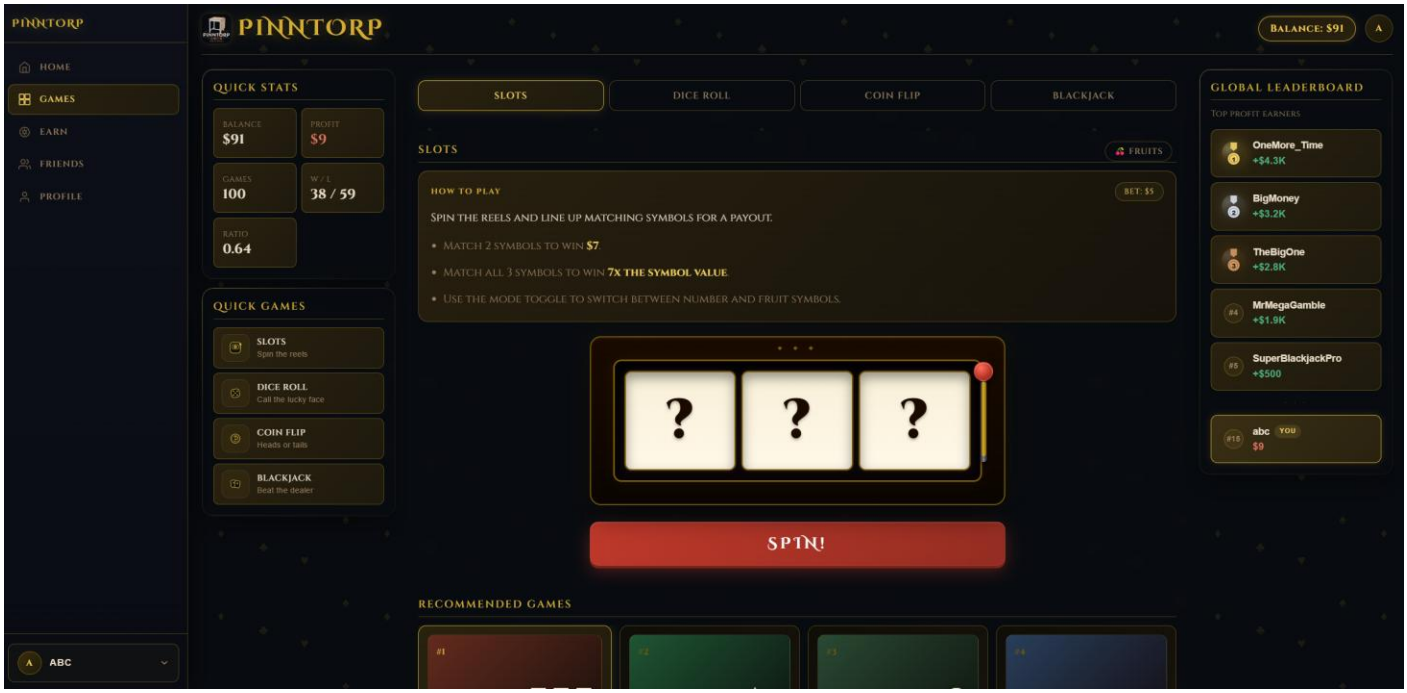

Figure 8: Games page with recommendation panel populated from friend activity

5.4 Earn Tab

The Earn tab features a casual clicker game which awards bonus balance to players who engage in longer game sessions, incentivizing them to spend more time gambling without betting more of their money. The limit of the clicker is set to $500 before the player is stopped.

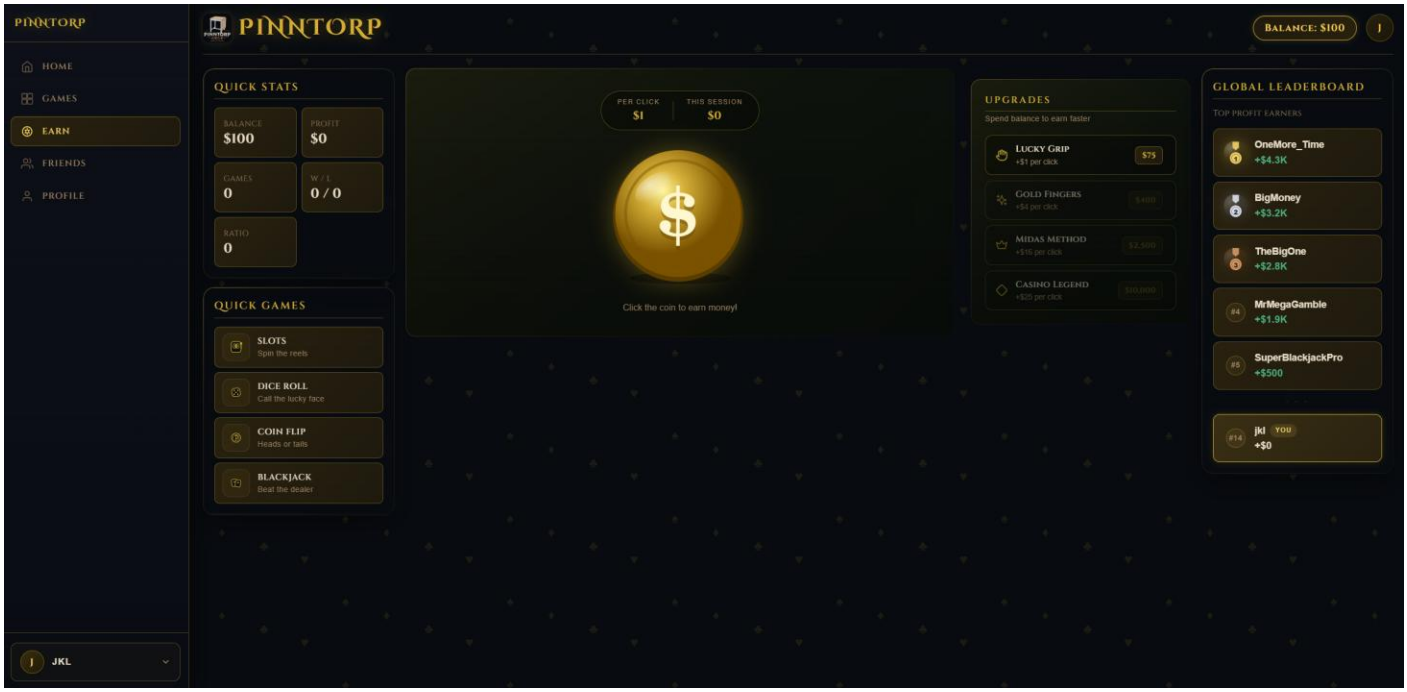

Figure 9: Earn / Clicker tab

5.5 Friends System

Players can search for other players using usernames, send friend requests, accept or reject received requests, and remove friends. The friends list is used by the recommendation system (ST-04, ST-05).

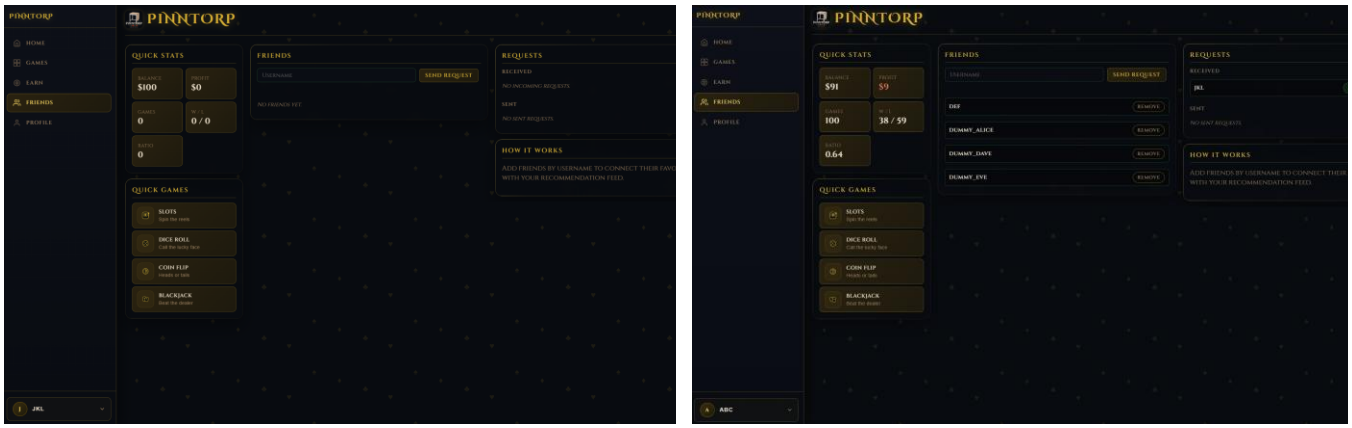

Figure 10: Friends tab empty (left) and with friends + pending request (right)

5.6 Profile & Game History

The profile page shows detailed statistics of the user and includes a list of game rounds completed in chronological order with game name, profit/loss delta, and time (ST-03, ST-08).

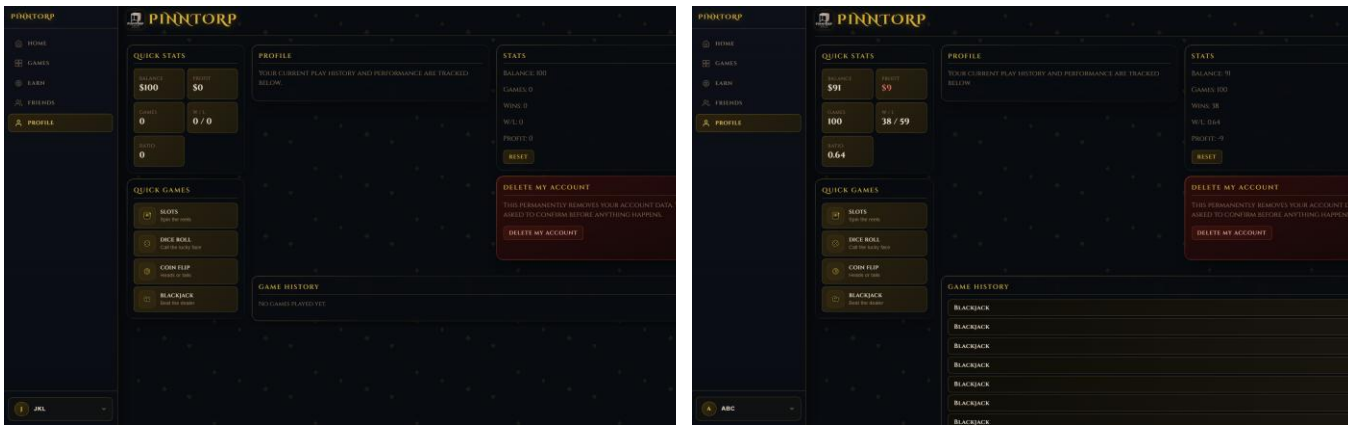

Figure 11: Profile tab with a fresh account (left) and with full game history and stats (right)

6. Iteration 3 Summary

6.1 Key Deliverables

Feature / Task	Assignee	Notes
Account deletion (frontend + /api/user/delete)	Adrian + Hayden	Completes TDD red-phase tests UT-10, UT-11, UT-12, ST-08, ST-09
Unified Java backend (single port 8080)	Hayden + Eric	Merges static file server, REST API, and WebSocket into one process
JWT authentication	Nikola + Eric	Replaced session-ID cookies with signed JWT tokens for all protected routes
Multiplayer Blackjack lobbies (WebSocket)	Eric	Server-authoritative game state; up to 4 players per lobby
Full automated test suite (58 tests)	Hayden	9 test files covering unit, integration, system, and regression tiers
Global leaderboard (/leaderboard endpoint)	Hayden	Top 10 users by profit; rendered on home page
Game history logging on profile page	Hayden	Every game round appended to user.history; rendered as a table on profile.html
Docker / CI/CD deployment	Eric	Dockerfile, docker-compose.yml, GitHub Actions publishing to ghcr.io
Architecture & API documentation	Eric + Nikola	docs/iter3/ARCHITECTURE.md and docs/iter3/API.md

6.2 Iteration 3 Burndown

The third iteration took place from March 24 to April 6, 2026. All 25 developer-days of core implementation tasks were successfully completed within this iteration resulting in the final iteration velocity value of 25 developer days per iteration (89% of total capacity 4 people * 28 days).

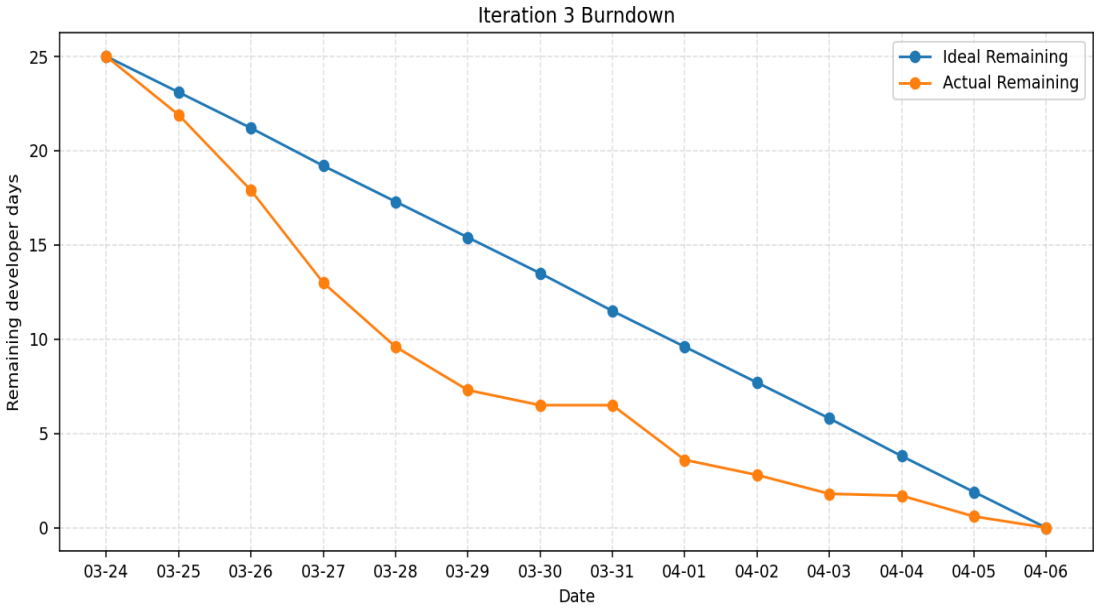

Figure 12: Iteration 3 Burndown Chart (Mar 24 to Apr 6, 2026)

7. Bonus Modifier Statement

PiNNTORP is eligible for the web-based bonus modifier since it uses the pure web-based architecture and can run entirely inside any modern browser without installing additional software on the user device.

Specifically, PiNNTORP:

- Is a Java-based web application that provides REST APIs and WebSockets over port 8080 and serves all static frontend resources (including HTML files, CSS stylesheets, and JavaScript code) from the same origin (server), which eliminates the need for CORS configurations, thus providing trivial hosting experience.
- Has a pre-built Docker container (ghcr.io/pinntorp-llc/pinntorp) for deployment to any cloud hosting or even a local PC, allowing to run the application from any modern web browser with no other installation required.
- All the application capabilities (gameplay, sign-in, friends management, game recommendations, leaderboard) are available from a pure browser experience without using any native application/client on the side of a user.
- Has a Hybrid Graceful Fallback design that ensures that the application can still be used (only locally) even when the server goes down, ensuring maximum uptime for users.